

## JS : Vérification d'un formulaire

Le JavaScript est un bon moyen d'effectuer une validation des données utilisateur avant l'envoi sur le serveur, c'est-à-dire côté client. Il faudra refaire une validation côté serveur, mais cela est une autre histoire dont vous ne vous occupez pas pour le moment.

### RÉCUPÉRATION DES ÉLÉMENTS

L'attribut `name` est indispensable dans la validation d'un formulaire. C'est sur cet attribut que l'on va se baser pour récupérer nos éléments.

Nous allons mettre un attribut « `name='nameForm'` » à notre formulaire. Côté HTML on aura alors :

```
<form name="nameForm">
```

Côté JavaScript, vous y accéderez comme ceci :

```
var myForm = document.nameForm;
```

Il est aussi possible d'utiliser les sélecteurs vus précédemment comme `getElementById`, `querySelector`, etc.

Pour récupérer la valeur d'un champ (`input`, `select`, etc.) on va aussi se baser sur son attribut `name`. Donc, si on considère un champ avec l'attribut « `name='nameInput'` » on accède à sa valeur comme ceci :

```
var myValue = document.nameForm.nameInput.value;
```

### LES ÉVÈNEMENTS LIÉS AUX CHAMPS

Avant de valider notre formulaire, en plus des éléments déjà contrôlés grâce aux attributs HTML, on va vérifier certaines informations dynamiquement lors du remplissage des champs. Et avertir l'utilisateur en cas d'erreur. On va utiliser des écouteurs d'événements, rappelez-vous lors d'un cours précédent on a utilisé un événement au clique sur un élément.

Pour les formulaires on peut utiliser un événement au « focus » d'un champ. C'est-à-dire tant que le champ reste sélectionné. Avec la fonction « `input` » :

```
nameInput.addEventListener("input", function(){  
    console.log(this.value)  
});
```

Ici on va donc vérifier la valeur du champ dans la console tant que celui-ci garde le « focus ».

Une autre possibilité est de vérifier cette valeur lorsque le champ perd le « focus ». Avec la fonction « `change` » :

```
nameInput.addEventListener("change", function(){  
    console.log(this.value)  
});
```

## LES EXPRESSIONS RÉGULIÈRES

Maintenant nous voulons contrôler plus en détail les données que l'utilisateur renseigne. Pour les champs dont on ne peut pas mettre d'attribut HTML, par exemple, qu'un champ contient uniquement des chiffres. Pour cela, on utilise les expressions régulières : **regex**.

Il faut alors déclarer une variable, dont la valeur est ce que l'on souhaite vérifier. On définit une expression régulière en plaçant son motif entre deux caractères « / ».

```
var regex = /@/; // la chaîne de caractère doit contenir le caractère @
```

Sa méthode `test` détecte la présence d'une correspondance avec le motif dans la chaîne de caractères passée en paramètre et renvoie `true` ou `false` selon le cas. Par exemple :

```
console.log(regex.test("adressemail.fr")); // renvoie false
console.log(regex.test("adresse@mail.fr")); // renvoie true
```

Voici une petite liste d'exemple de **motif** possible :

- **abc** La chaîne contient "abc"
- **[abc]** La chaîne contient soit "a", soit "b", soit "c"
- **[a-z]** La chaîne contient n'importe quelle lettre minuscule de l'alphabet
- **[0-9]** La chaîne contient un chiffre
- **a.c** La chaîne contient "a" suivi d'un caractère (n'importe lequel) suivi de "c"
- **a\.c** La chaîne contient "a.c"
- **a.+c** La chaîne contient "a" suivi d'un ou plusieurs caractères (n'importe lesquels) suivi de "c"
- **a.\*c** La chaîne contient "a" suivi de zéro ou plusieurs caractères (n'importe lesquels) suivi de "c"

Il existe de nombreux autres types de motif possible.

Voici une explication de comment fonctionne ces motifs :

- Les crochets « [] » définissent un intervalle de caractères. Toute chaîne contenant au moins un caractère dans cet intervalle correspondra au motif. Les motifs **[a-z]** et **[A-Z]** permettent de rechercher n'importe quelle lettre de l'alphabet, respectivement en minuscules ou en majuscules.
- Le caractère point « . » permet de remplacer n'importe quel caractère.
- Le caractère « \ » ("antislash" ou "backslash") indique que le caractère qui suit doit être recherché en tant que tel.
- Le caractère « + » permet de rechercher une ou plusieurs occurrences de l'expression qui le précède.
- Le caractère \* permet de rechercher zéro ou plusieurs occurrences de l'expression qui le précède.

On peut donc obtenir un motif complexe comme : `/.+@.+\..+/`

Décodons ce motif :

- Commence par un ou plusieurs caractères ( `.+` )
- Contient ensuite le caractère « @ » ( `@` )
- Contient ensuite un ou plusieurs caractères ( `.+` )
- Contient ensuite le caractère « . » ( `\.` )
- Finit par un ou plusieurs caractères ( `.+` )

Il est donc pratique pour vérifier une adresse mail.

## VÉRIFICATION DU FORMULAIRE

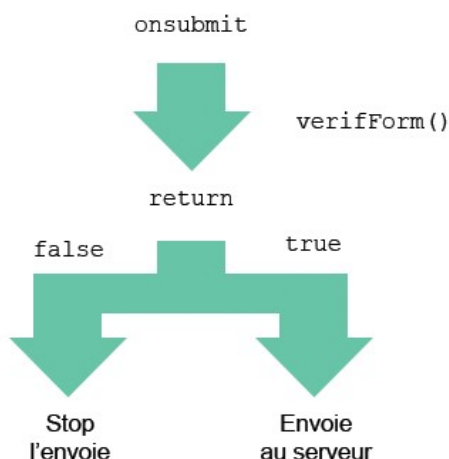
Au clique sur le bouton « envoyer » de notre formulaire, on peut faire une vérification de celui-ci, avant d'envoyer les données. On aura donc besoin d'un nouvel attribut `onsubmit` que l'on va placer directement sur notre formulaire. Du côté HTML on écrit alors :

```
<form name="contactForm" onsubmit="return verifForm()">
```

La fonction `return` va nous permettre de stopper l'envoi du formulaire si une information est inexacte ou une condition non remplie. Comme le montre ce schéma :

```
<form name="contactFrom" onsubmit="return verifForm()"
```

A blue rectangular box representing a form. Inside the box, there are two white rectangular input fields stacked vertically. Below the input fields, centered, is a dark blue button with the word "SUBMIT" in white capital letters.



Ensuite, dans notre JavaScript on va créer la fonction de validation qui est appelée par notre attribut `onsubmit` :

```
function verifForm() {  
    // instructions avant validation  
}
```

C'est dans cette fonction qu'on mettra, lors d'une erreur rencontrée :

```
return false;
```

Dès que cette instruction est donnée, le formulaire va stopper l'envoi des informations.

Par exemple :

```
if(champObligatoires.value==""){  
    return false;  
}
```

## MESSAGE DE VALIDATION

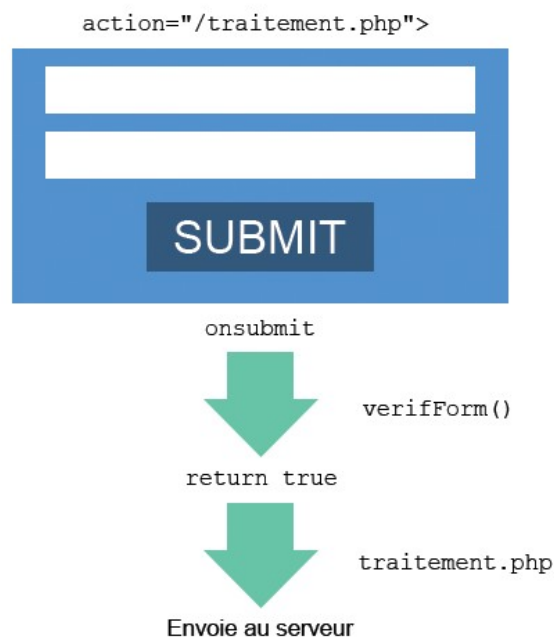
Tout comme pour les erreurs il est important de notifier à l'utilisateur que le formulaire a bien été rempli et envoyé. Pour cela on va ajouter dans notre fonction, une nouvelle condition avec la méthode `checkValidity()` par exemple :

```
if (contactForm.checkValidity()){  
    // instructions d'affichage du message de validation  
}
```

## TRAITEMENT DES DONNÉES

Pour le traitement des données et l'envoi au serveur, on va avoir besoin de rediriger vers une autre page. On aura besoin de l'attribut `action` qui donne le chemin du nouveau fichier (en php) qui traitera les données. Côté HTML on va donc rajouter :

```
<form name="contactForm" onsubmit="return verifForm()" action="/traitement.php">
```



On peut tout à fait laisser le navigateur gérer les contraintes HTML natives (`required`, `type="email"`, `pattern`, etc.) mais si l'on souhaite un formulaire personnalisé, il est possible de lui ajouter en HTML l'attribut `novalidate`, comme ceci :

```
<form id="signup-form" novalidate>
```

Dans ce cas, il faudra gérer en JavaScript toutes les contraintes souhaitées pour valider le formulaire.

Il est également possible de laisser gérer les contraintes HTML natives au navigateur, et d'ajouter en JavaScript uniquement les validations métier qui ne peuvent pas s'exprimer en HTML (adresse mail et mot de passe identiques à leur confirmation).

Imaginons un formulaire d'inscription avec seulement un champ mail et un champ mot de passe à remplir, comme dans l'exemple ci-dessous :

```
<body>
  <form id="signup-form">
    <div>
      <label for="telephone">Téléphone :</label>
      <input id="telephone" name="phone" type="tel" pattern="^\+?[0-9\s]{10,15}$">
    </div>

    <div>
      <label for="email">Email *</label>
      <input type="email" id="email" name="mail" required>
      <div class="error"></div>
    </div>

    <div>
      <label for="email-confirm">Confirmer Email *</label>
      <input type="email" id="email-confirm" name="mailConfirm" required>
    </div>

    <div>
      <label for="mdp">Mot de passe *</label>
      <input type="password" id="mdp" name="password" required
        pattern="^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[!@#$%^&*~_]).{8,}$"
        title="8 caractères minimum, avec une majuscule, une minuscule, un chiffre et un caractère spécial">
      <div class="error"></div>
    </div>

    <div>
      <label for="mdp-confirm">Confirmer Mot de passe *</label>
      <input type="password" id="mdp-confirm" name="passwordConfirm" required>
    </div>

    <p>Les champs marqués d'un astérisque * sont obligatoires</p>

    <button type="submit">Envoyer</button>
  </form>

  <script src="script.js"></script>
</body>
```

La confirmation du mail et du mot de passe peut se faire directement dans le fichier script.js, comme une première couche de vérification frontend avant l'envoi de données au backend (cela n'enlève pas la validation côté serveur, qui est toujours le système vérifiant chaque risque de manques de données ou de failles de sécurité).

- Pour cela, en JavaScript, il va falloir vérifier que les champs soient tous remplis, et les comparer entre eux.
- De plus, il est souhaitable d'afficher l'erreur si les champs ne correspondent (erreur qui ne sera pas prise en compte par le navigateur).
- Enfin, il faudra préparer la donnée propre dans un objet prêt à être envoyé au backend.

```
const signupForm = document.querySelector("#signup-form")

signupForm.addEventListener("submit", function (event) {

  event.preventDefault();
```

Tout d'abord, ajouter un écouteur d'évènement de type "submit" permet d'écouter la validation du formulaire.

Ensuite, empêcher le comportement par défaut de la validation du formulaire se fait grâce à l'utilisation de la méthode .preventDefault() de l'objet "évènement" (si aucun attribut action n'est défini et qu'aucun gestionnaire onSubmit n'empêche la soumission, la page va se recharger sans rien faire).

Une fois les vérifications faites, on peut préparer la donnée (les réponses de l'utilisateur remplies dans les champs appropriés) :

```
const formData = new FormData(signupForm);  
  
const values = Object.fromEntries(formData.entries());  
  
// L'objet values est ensuite envoyé au backend  
// ...  
:      :
```

- new FormData() crée un objet FormData contenant tous les champs du formulaire et leurs valeurs actuelles
- formData.entries() produit un itérateur de paires [clé, valeur], ce qui est attendu pour un objet, et qui sera propre pour que le backend puisse le lire et le comprendre correctement (chaque clé sera le name du champ, et chaque valeur la réponse renseignée par l'utilisateur)
- Object.fromEntries() transforme ces paires en objet JavaScript

## ATTENTION

Si le formulaire contient des champs pouvant avoir plusieurs valeurs, comme les input de type checkbox, il faudra renvoyer toutes les valeurs associées sous forme de tableau, comme ceci :

```
const formData = new FormData(signupForm);  
  
const values = {  
  ...Object.fromEntries(formData.entries()),  
  hobbies: formData.getAll("hobbies")  
};
```

Dans cet exemple, on récupère toutes les données transformées en objet JS, dont la clé 'hobbies' qui aura comme valeur un tableau regroupant toutes les valeurs sélectionnées par l'utilisateur.